

Slovenská Technická Univerzita, Bratislava
Fakulta Informatiky a Informačných Technológií

Elektronický obchod - semestrálny projekt

Základy webových technológií

11.5.2025
Bratislava

Terézia Kliešťiková
Alena Múková

Obsah

Obsah	2
Zadanie	3
Databáza	3
E-R dátový model	3
Fyzický dátový model	4
Zmeny v dátovom modeli oproti fáze 1	4
Návrhové rozhodnutia	5
Opis implementácie vybraných prípadov použitia	7
Zmena množstva pre daný produkt	7
Prihlásenie	7
Vyhľadávanie	8
Pridanie produktu do košíka	8
Stránkovanie	9
Základné filtrovanie	10
Preusporiadanie produktov	10
Admin – pridanie produktu	10
Admin - odstránenie produktu	10
Admin - úprava produktu	11
Snímky obrazoviek	11
Používateľská príručka (Windows / MacOS)	17
Používateľská príručka na zakúpenie produktu	18
Záver	18
Dôvod 2. odovzdania	19

Zadanie

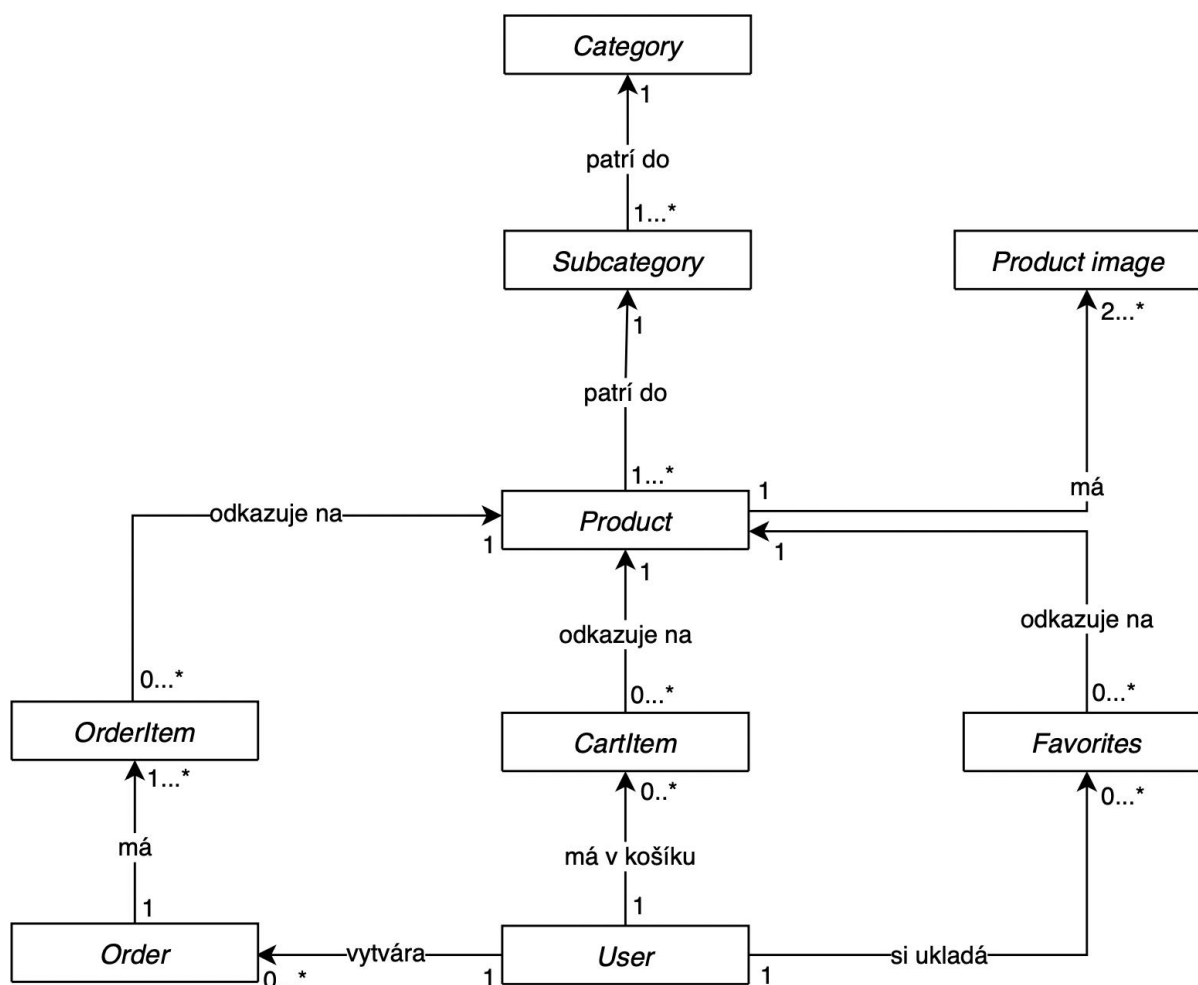
Našou úlohou bolo vytvoriť vlastnú webovú aplikáciu – e-shop, ktorá komplexne rieši typické prípady použitia elektronického obchodu. Aplikácia je vytvorená vo dvojici a pozostávať z klientskej aj administrátorskej časti. V rámci klientskej časti sme implementovali funkcie ako prehľad produktov s filtrovaním a stránkovaním, detail produktu, nákupný košík, registráciu, prihlásenie a možnosť uskutočniť objednávku. Administrátorská časť obsahuje správu produktov (vytváranie, úprava, mazanie).

Projekt bol vypracovávaný v 2 hlavných častiach. V prvej časti sme tvorili návrh používateľského rozhrania v podobe skíc v nástroji Figma a návrh dátového modelu. Z týchto skíc sme následne vytvorili HTML, CSS a JavaScript šablóny ktoré sú plne responzívne. V druhej fáze projektu sme aplikovali PHP framework Laravel, ktorý využíva server-side rendering. Vďaka tomu sa nám podarilo vytvoriť plne funkčný interaktívny e-shop.

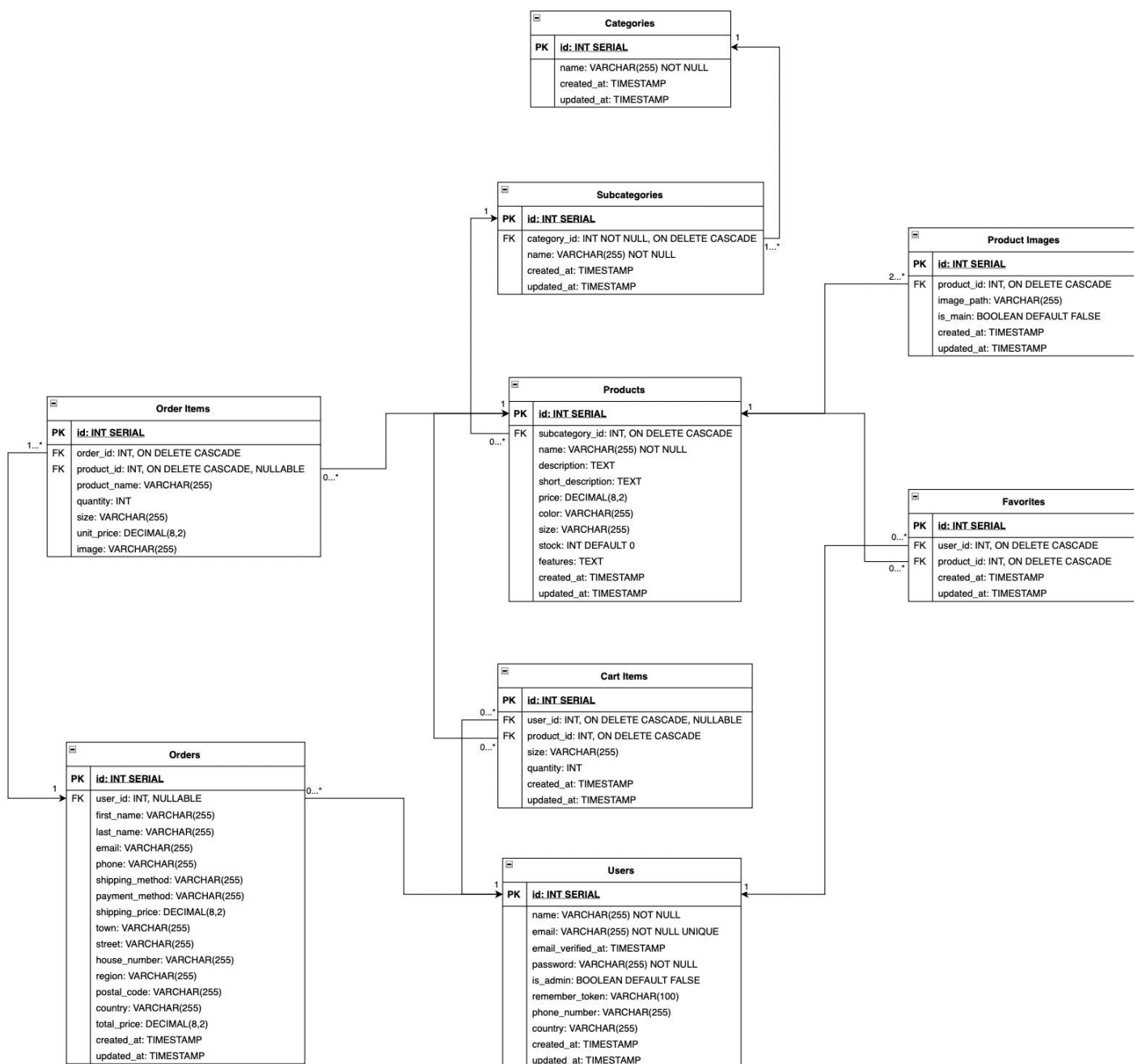
Databáza

Pracujeme s databázovým systémom PostgreSQL.

E-R dátový model



Fyzický dátový model



Zmeny v dátovom modeli oproti fáze 1

Pridanie tabuliek **Category** a **Subcategory** umožnilo lepšiu organizáciu produktov pričom má používateľ možnosť filtrovať produkty aj podľa týchto entít. Tabuľky **Payment** a **Shipping** sme zahrnuli do tabuľky **Orders**, čím sme zabezpečili, že všetky dôležité informácie o objednávke sú v jednej entite a nemusíme zbytočne využívať JOIN (spájanie tabuliek). Rozhodli sme sa odstrániť tabuľku **Search History**, pretože sme ukladanie histórie vyhľadávania neimplementovali – bolo by to zbytočné kvôli komplexite a nenašli sme tomu praktické využitie.

V tabuľkách pribudlo viac atribútov: Tabuľke **Products** pribudol textový atribút short_description, pretože sa v detaile produktu tento textový atribút zobrazuje a musí byť priradený konkrétnemu produktu. Admin zároveň musí byť schopný zmeniť short_description nezávisle od základej description. Tabuľke **Users** pribudli atribúty name, phone_number, country na uchovanie základných informácií o používateľovi a kvôli automatickému vyplneniu dodacích údajov k objednávkam. Pribudli aj časový atribút email_verified_at a atribút remember_token. Tabuľke **OrderItem** pribudol atribút product_name, ktorý slúži najmä na zobrazenie objednávok v profile používateľa. Tabuľka **Product Image** má zmenený atribút name na image_path pretože meno obrázka je nepotrebné, potrebujeme ukladať cestu k nemu. Taktiež pribudol aj atribút is_main, ktorý určuje, či je tento obrázok hlavným obrázkom produktu (ten sa prioritne zobrazuje v liste produktov).

Tabuľkám **CartItem**, **Favorites**, **Product Image** pribudli časové atribúty created_at, updated_at.

Návrhové rozhodnutia

Pri vývoje webovej aplikácie sme si vybrali **PHP framework Laravel**, ktorý poskytuje robustný základ pre vývoj webových aplikácií a bol odporúčaný na prednáškach. Vďaka Laravelu sme zvládli vytvoriť backendovú logiku bez potreby pridávania externých knižníc.

Zvolili sme si použitie databázového systému **PostgreSQL**, keďže s týmto systémom máme skúsenosť z predmetu Databázové systémy a dokázal pokryť všetky naše požiadavky.

Pri návrhu používateľského rozhrania sme využili populárny **CSS framework – Bootstrap**, ktorý nám poskytol preddefinované komponenty a responsívny grid systém vďaka čomu sme nemuseli definovať niektoré štýly samé od základov. Bootstrap je do projektu pridaný prostredníctvom CDN linku v hlavičke (head) layoutu.

Obrázky sú zobrazované v Blade šablónach pomocou HTML tagu `<img>` a generujú sa dynamicky podľa src atribútu na základe cesty uloženej v databáze. Zvolili sme prístup, že v databáze máme pre každý obrázok uloženú iba cestu k súboru, nie binárny obsah súboru, čo zabezpečuje menšiu záťaž databázy. Samotné obrázky sa potom nachádzajú v priečinku projektu `/public/images`.

Keďže vytvárame fiktívny eshop pre účely predmetu WTECH a nevlastníme reálne produkty, čiže ani ich fotky, produkty sme čerpali z iných webových stránok alebo z open-source zdrojov z internetu. Chceli sme mať jednotný dizajn fotiek produktov. Nepodarilo sa nám nájsť dostatočne veľa open-source fotiek produktov do nášho e-shopu tak, aby bola medzi fotkami zachovaná konzistentnosť, a preto sme sa rozhodli použiť obrázky produktov z verejne dostupných stránok na internete. Po dôkladnom zvážení a rešpektovaní rozsahu použitia - nekomerčné a čisto študijné účely sme obrázky čerpali zo stránok pukassurfshop.com, mctavish.com, decathlon.sk, surflocos.com. Tieto obrázky sme zvolili pre ich vizuálnu kvalitu a estetiku. Sme si vedomé, že tieto obrázky podliehajú autorským

právam, a preto by boli v reálnom projekte nahradené vlastnými alebo licencovanými obrázkami.

Rozhodli sme sa neimplementovať komplexný **systém oprávnení a rolí**, keďže v našom eshope existuje iba zákazník a admin. Použitie plnohodnotného laravelovského systému pre roly a oprávnenia by zbytočne zvýšilo zložitosť riešenia. Usúdili sme, že v našom prípade je dostatočné rozlišovať admin používateľa na základe atribútu `is_admin = True` v databáze v tabuľke `users`. Bežný používateľ má prístup do eshopu, môže si prehliadať produkty a nakupovať. Admin používateľ nevie nakupovať produkty – vie si ich iba prezerať. Admin má však oprávnenie cez adminské rozhranie do ktorého sa vie prihlásiť upravovať, mazať alebo pridávať nové produkty.

Registrácia a prihlasovanie používateľov využíva Laravel autentifikačnú vrstvu – rozhranie `Authenticatable` – ktoré zabezpečuje bezpečnú správu identity používateľov a šifrovanie hesiel. Rozlišovanie medzi zákazníkom a adminom je riešené cez vlastný boolean atribút `is_admin`.

Prístup do adminského rozhrania sme sa rozhodli realizovať oddeleným prihlasovacím rozhraním, ktoré nie je verejne viditeľné ani odkazované zo zákazníckej časti aplikácie. Prístup je možný iba cez URL `/login/admin`. Na tejto adrese je sprístupnený prihlasovací formulár, ktorý umožňuje prihlásenie iba administrátorom (používateľia, ktorí majú `is_admin = True`). Po úspešnom prihlásení je admin presmerovaný do administrátorského rozhrania, kde má prístup ku správe produktov. Toto riešenie minimalizuje viditeľnosť admin loginu (v budúcnosti by bolo vhodné ešte viac zatajiť URL na admin login) a zabezpečuje prístup iba adminom so správnymi prihlasovacími údajmi.

Pri **validácii údajov v nákupnom košíku** sme sa rozhodli pre riešenie na strane klienta pomocou JavaScriptu. Tento prístup nám umožnil okamžitú spätnú väzbu používateľovi bez potreby obnovovania stránky alebo volania servera. JavaScriptom sme kontrolovali povinné polia, správnosť emailovej adresy, číselné hodnoty pre telefón, PSČ a číslo domu, ako aj formát platobných údajov. V prípade platby kartou sme implementovali vlastnú validáciu dátumu expirácie, formátu čísla karty, mena na karte a CVC kódu.

Dôvodom pre túto voľbu bola jednoduchosť implementácie, zvýšenie používateľského komfortu a lepšia interaktivita formulárov. Vzhľadom na to, že ide o školský projekt bez reálnej integrácie platobných brán alebo externých služieb, nepovažovali sme za potrebné pridávať serverovú validáciu vo všetkých krokoch. V reálnom nasadení by sme samozrejme doplnili aj validáciu na strane servera pre zabezpečenie integrity a ochranu pred manipuláciou.

Aj keď má používateľ v našej aplikácii na výber viaceré spôsoby platby ako napríklad Apple Pay, PayPal, či GooglePay, tieto možnosti v projekte nie sú reálne implementované. Ich výber slúži len na demonštráciu používateľského rozhrania a možného budúceho rozšírenia.

V produkčnom nasadení by sme tieto platobné brány prepojili s príslušnými API a zabezpečili by sme celý proces ich integrácie a overovania.

Opis implementácie vybraných prípadov použitia

Zmena množstva pre daný produkt

Používateľ môže zmeniť množstvo konkrétneho produktu priamo v košíku prostredníctvom `<input type="number" name="quantity">`. Tento prvok je umiestnený vo formulári, ktorý sa automaticky odosiela pri zmene hodnoty cez `onchange="this.form.submit()"`. Používateľ si tiež môže vybrať množstvo daného produktu priamo v detaile produktu `product_detail.blade.php` kde sa pri zmene hodnoty aktualizuje formulár cez `on_change = "updateQuantity()"`. Metóda je definovaná v Javascripte `product_details.js`. Dôležitou vlastnosťou formulára je aj `@method = "PUT"`, pomocou ktorej aktualizujeme hodnoty existujúceho zdroja – teda nášho `form`.

Formulár smeruje na route definovanú v `routes/web.php`:

```
Route::put('/cart/update/{id}', [CartController::class, 'updateCartItem'])->name('updateCartItem');
```

Spracovanie zmeny sa vykonáva v metóde `updateCartItem()` v `CartController`.

- **Pre prihláseného používateľa** sa zmení záznam v databáze v tabuľke **CartItems** - nájde sa položka podľa `user_id` a `product_id`, následne sa aktualizuje veľkosť (`size`) a množstvo (`quantity`).
- **Pre neprihláseného používateľa** sa aktualizuje hodnota v session `session()->get('cart')` a nová verzia košíka sa uloží späť `session()->put('cart', $cart)`.

Následné zobrazenie položiek košíka sa vykoná v metóde `step1()` v rovnakom kontroléri.

Táto metóda načíta buď:

- dáta z databázy (ak je používateľ prihlásený), alebo
- zo session (ak nie je prihlásený),

a odošle ich do Blade šablóny `shopping_cart1.blade.php`.

Prihlásenie

Používateľ sa do svojho konta v eshope prihlasuje cez rozhranie vytvorené v blade šablóne `login.blade.php` (admin sa prihlasuje cez `admin_login.blade.php` a prihlásenie spracováva `AdminController` vo funkcií `login()`). Používateľ vidí dva formuláre:

Registračný formulár:

Formulár obsahuje polia pre meno, email, telefónne číslo, krajinu, heslo a potvrdenie hesla. Odsiela sa POST požiadavkou a je zabezpečený pomocou csrf. Keď používateľ vyplní všetky polia formulára, tak formulár sa odosiela buttonom Register. Samotná registrácia sa spracováva v `UserController`i vo funkcií `register`, ktorá validuje vstupné údaje pomocou `validate()` kde sa kontroluje jedinečnosť emailu a správnosť a zhoda vytvoreného hesla.

Používateľ sa zapisuje do databázy do tabuľky **users** cez `User::create()` spolu s heslom, ktoré je zahashované funkciou `Hash::make()`. Nový registrovaný používateľ je ihneď prihlásený do svojho konta pomocou laravel funkcie `Auth::login()` a presmerovaný na hlavnú stránku. V prípade nesprávnych registrčných údajov je používateľ presmerovaný späť a zobrazia sa mu aj chyby, ktoré nastali pri validácii.

Prihlasovací formulár:

Login formulár obsahuje polia pre email a heslo. Ak pri validácii údajov nastanú chyby, tie sa následne taktiež zobrazia ako chybové hlášky. Formulár sa odosiela POST požiadavkou. A následne sa spracováva v `UserController`i vo funkcii `login`. Po odoslaní vykonám kontrolu, či sú polia vzplnené, a či sa snaží prihlásiť bežný používateľ (tento formulár nedovoľuje prihlásenie adminovi). Následne sa vykoná kontrola správnosti prihlasovacích údajov pomocou laravel funkcie `Auth::attempt()` a v prípade správnych prihlasovacích údajov je používateľ prihlásený do svojho konta a presmerovaný na homepage. Ak je prihlásenie neúspešné, zobrazí sa chybová hláška s touto informáciou.

Vyhľadávanie

Používateľ dokáže vyhľadávať produkty podľa mena. Vyhľadávací formulár sa nachádzav blade šablónach `homepage.blade.php` a `products.blade.php`. (Ďalšie vyhľadávanie mám aplikované v adminovom rozhraní, kde admin dokáže vyhľadávať produkty, ktoré bude potrebuje na úpravu. Adminovské vyhľadávanie produktov funguje podobne ako vyhľadávanie bežného používateľa.)

Vyhľadávací formulár posieľa GET požiadavku s parametrom 'search'. Vyhľadávanie prebieha už v rámci aktuálnej URL adresy napr. `/products?search= ...` Vyhľadávanie spracováva `ProductController` v metódach `index()`, `byCategory()` alebo `bySubcategory()`. Vo všetkých metódach je vyhľadávanie implementované rovnako. Kontrolujem, či bol parameter `search` vyplnený. Ak áno, tak na query ktorá bude vyhľadávať vyhovujúce produkty v tabuľke **products** ktoré sa napokon používateľovi zobrazia aplikujem filter podľa `name` stĺpca s použitím `ILIKE`, ktoré je case-insesitive fulltext vyhľadávanie v PostgreSQL databáze.

Pridanie produktu do košíka

Používateľ môže pridať produkt do nákupného košíka priamo z prehľadu produktov alebo zo stránky detailu produktu. Táto funkcionalita využíva HTTP metódu POST, keďže ide o vytvorenie novej položky v košíku

Pridanie z katalógu produktov `products.blade.php` :

Zoznam produktov zobrazuje pre každý produkt ikonu na pridanie alebo odstránenie z košíka. Ak produkt ešte nie je v košíku, zobrazí sa formulár s POST požiadavkou. Ak je používateľom admin, tak sa mu ikona na pridanie a odstránenie produktu do košíka nezobrazí. Admin si teda produkt zakúpiť nemôže.

Pridanie z detailu produktu `product_detail.blade.php` :

Na stránke detailu si používateľ môže zvoliť veľkosť a množstvo. Formulár sa po každej zmene aktualizuje pomocou `onchange="updateSize()"` a `onchange="updateQuantity()"`. Metódy sú definované v Javascripte `product_details.js`. Po stlačení `<button type="submit" form="add-to-cart-form">` sa odošle formulár s POST požiadavkou. Ak je používateľom admin, tak sa mu button na pridanie produktu do košíka nezobrazí. Admin si teda produkt zakúpiť nemôže.

Formulár smeruje na route definovanú v `routes/web.php` :

```
Route::post('/cart/add/{id}', [CartController::class, 'addToCart'])->name('addToCart');
```

Metóda `addToCart` **v kontroléri** `CartController` :

Ak je používateľ prihlásený, hľadáme existujúci záznam o danom produkte v košíku používateľa priamo cez databázu. Ak daný záznam už existuje, zvýši sa počet tohto produktu o 1 a ak produkt neexistuje, tak sa tento produkt vloží do databázy do tabuľky **CartItem**s cudzím kľúčom na používateľa, cudzím kľúčom na daný produkt, s jeho vybranou veľkosťou a počtom kusov.

Ak používateľ prihlásený nie je, pracujeme so sessions. Pomocou `session()` - `>get('cart')` vyberieme aktuálne položky v košíku. Ak sa tam daný produkt už nachádza, zvýšime jeho počet o 1, a ak sa tam ešte nenachádza, pridá sa nová položka do session s cudzím kľúčom na produkt a jeho vybranou veľkosťou a počtom kusov. Cez `session()->put('cart', $cart)` sa vloží daný záznam do session.

Stránkovanie

Stránkovanie aplikujem pri prezeraní produktov v blade šablóne `products.blade.php` v časti `<div class="strankovanie">` (v admin profile je stránkovanie aplikované taktiež). Stránkovanie sa spracováva v `ProductController` v metódach `index()`, `byCategory()` alebo `bySubcategory()`. Vo všetkých metódach je stránkovanie implementované rovnako – pomocou Laravel metódy `paginate(12)` ktorá automaticky zobrazí produkty do stránok po 12 produktov. 12 sme zvolili kvôli príjemnosti prehliadania produktov a taktiež kvôli tomu, aby sa dalo stránkovanie aplikovať pre ukážku aj na menší počet produktov. V budúcnosti by sa s touto hodnotou dalo jednoducho hýbať. Vo view pracujem s laravel metódami `currentPage()`, `lastPage()`, `hasMorePages()`, `onFirstPage()`, vďaka ktorým dokážem efektívne pracovať s jednotlivými komponentami vo view a pridať im funkčnosť zmeny stránky tak ako potrebujeme. Pri presune na novú stránku zachovávam všetky aktuálne nastavené filtre, zoradenia a vyhľadávania v URL pomocou `request()->fullUrlWithQuery(...)` .

Základné filtrovanie

Naša webová aplikácia umožňuje filtrovanie zobrazených produktov na základe ceny, subkategórie farby a veľkosti. Rozhranie pre filtrovanie sa nachádza vo view `products.blade.php`. Filtrovací formulár odosiela filtrové dáta cez URL a GET požiadavku. Vo filtry sa dá zvoliť cenový rozsah pomocou 2 `<input type="range">`. Subkategória, farba a veľkosť sa volia pomocou `<input type="checkbox">`. Filtrovanie sa spúšťa tlačidlom Apply filter.

Filtrovanie sa spracováva v `ProductController` funkciou `applyFilter()`. Vstupom je `$request` (GET dáta) a `$query` na ktorú sa filtrovanie bude aplikovať. Vo funkcií sa postupne kontroluje subkategória, farba a veľkosť pomocou `whereIn` a `where`. Cena sa kontroluje a aplikuje pomocou `whereBetween`. Výsledná `$query` potom pokračuje ďalej do funkcie, ktorá zobrazuje stránku s produktami.

Vďaka javascriptu v `products.js` je filter interaktívny. Pri načítaní stránky sa v blade šablóne automaticky predvyplnia checkboxy/slider hodnoty na základe `value = request()`.

Preusporiadanie produktov

Vo view `products.blade.php` má používateľ možnosť preusporiadať si zobrazenie produktov podľa ceny alebo abecedy. Túto funkcionality implementuje úsek kódu v `<div class="dropdown">`. Rozbaľovacie tlačidlo s textom Sort by otvára zoznam možností.

Každá možnosť je odkaz, ktorý aktualizuje URL parametre. Toto sa spracováva v `ProductController` vo funkcií `applySorting()`, ktorá číta parameter 'sort' a na základe toho sa vykoná `orderBy()`.

Admin – pridanie produktu

Keď je admin prihlásený vo svojom profile, po kliknutí na Add product button vo view, ktoré je v `admin_profile.blade.php` sa mu zobrazí overlay s formulárom na pridanie nového produktu. Formulár obsahuje polia - niektoré sú povinné a bez nich formulár nepôjde odoslať. V `admin_profile.js` sa validujú polia ešte pred odoslaním formulára a poskytuje adminovu spätnú väzbu v reálnom čase. Ak vyplní všetky povinné polia, tak sa odblokuje button na odoslanie formuláru. Samotné pridanie produktu sa vykonáva v `AdminController` vo funkcii `add()`. Každý produkt je najskôr validovaný a následne uložený do databázy - do tabuľky **products** čím predchádzame chybám. Obrázky sa fyzicky ukladajú do `public/images/products/` a ich názov sa zapisuje do tabuľky **product_images**.

Admin - odstránenie produktu

V administrátorskom rozhraní (`admin_profile.blade.php`) je pri každom produkte tlačidlo Delete. Po kliknutí na tlačidlo sa zobrazí potvrdzovacie okno (`deleteOverlay`), v ktorom musí admin tlačidlom Yes potvrdiť vymazanie produktu z eshopu. To odošle DELETE požiadavku na URL adresu, ktorá je dynamicky nastavená pomocou `admin_profile.js` podľa ID produktu, ktorý sa vymazáva. Server spracováva požiadavku v `AdminController` v metóde `delete()`, ktorá vymaže všetky obrázky produktu z databázy z tabuľky

product_images, ktoré patria tomuto produktu a potom aj samotný produkt z **products** tabuľky.

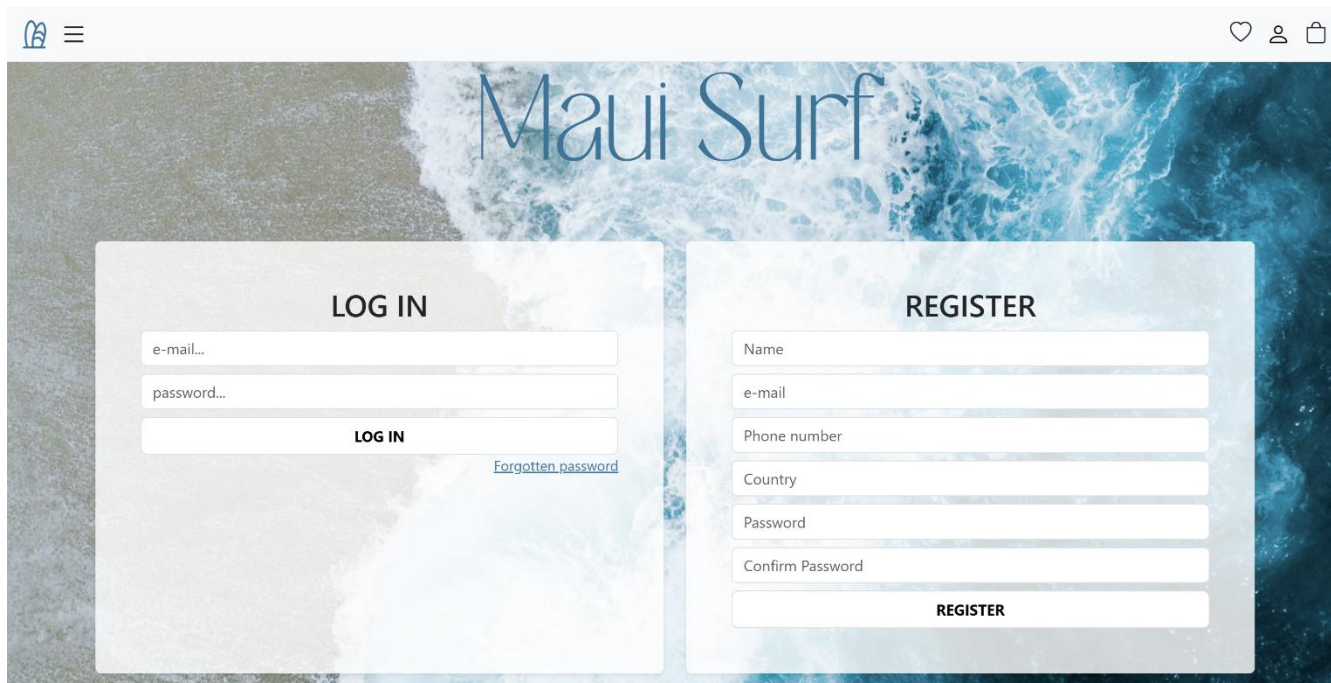
Rozhodli sme sa, že pri vymazávaní produktu, s čím súvisí vymazávanie príslušných obrázkov nebudeme vymazávať obrázok aj fyzicky z priečinka ale iba záznam z databázy. Toto rozhodnutie sme spravili preto, lebo máme nedostatok obrázkov a niektoré obrázky sú priradené viacerým produktom.

Admin - úprava produktu

V administrátorskom rozhraní (`admin_profile.blade.php`) je pri každom produkte tlačidlo Edit. Po kliknutí na tlačidlo sa vďaka `admin_profile.js` načíta formulár so všetkými predvyplnenými údajmi pre daný produkt. Administrátor následne môže zmeniť tieto údaje. Zabezpečujeme, že produkt musí mať vždy minimálne 2 fotografie. Po odoslaní PUT požiadavky sa formulár spracováva v `AdminController` vo funkcii `edit()`. Prebehne validácia údajov a zmeny v databáze sa vykonajú iba pre tie atribúty, ktoré sú nastavené správne. V opačnom prípade na týchto atribútoch úprava neprebehne.

Snímky obrazoviek

PRIHLÁSENIE ZÁKAZNÍKOV

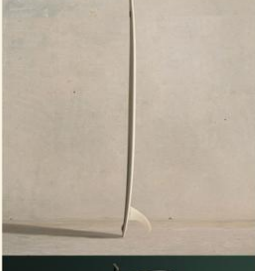


The screenshot displays the Maui Surf website's user interface. At the top, there is a navigation bar with a logo on the left and icons for a heart, a person, and a shopping bag on the right. The main background features a large, stylized image of ocean waves with the text "Maui Surf" overlaid in a light blue font. Below this, there are two prominent white boxes for user authentication. The left box is titled "LOG IN" and contains two input fields labeled "e-mail..." and "password...", followed by a "LOG IN" button and a link for "Forgotten password". The right box is titled "REGISTER" and contains five input fields labeled "Name", "e-mail", "Phone number", "Country", and "Password", followed by a "Confirm Password" field and a "REGISTER" button.

The image shows a web application for 'Maui Surf'. The top navigation bar includes a logo, a search bar, and icons for a heart, a person, and a shopping bag. Below the navigation bar, the main content area is divided into three sections: 'SURFBOARDS', 'EQUIPMENT', and 'ACCESSORIES'. The 'SURFBOARDS' section is currently active, displaying a grid of 12 surfboard products. Each product is shown in a card format with an image of the surfboard, its name, and its price. The surfboards are: Wave rider Pro (\$499.99), Wave Hunter (\$399.99), ABC rider (\$199.99), StormSurge (\$899.99), SunDrifter (\$349.99), Reef Rider (\$1,999.99), Ocean pulse (\$1,199.99), BlueEcho (\$783.00), SaltSprint (\$1,299.99), SeaZen (\$199.99), DriftNovan (\$399.99), and Maui go (\$1,399.99). Each card also features a heart icon and a shopping bag icon. At the bottom of the grid, there is a pagination control with a left arrow, a '1' button, a '2' button, and a right arrow. The footer of the page contains contact information, customer service links, and social media icons.

DETAIL PRODUKTU





Wave rider Pro

★★★★★ Reviews

Fast and responsive shortboard for your daily surfing sesh

499.99\$

 [Buy](#)

Features ▾

- Carbon fiber rails.
- Ti-fn setup.

Description ▾

High-quality surfboard for experienced surfers.

Delivery & Returns ▾

Reviews ▾

We think you will like



CONTACT

Email: example@email.com
Phone: +123 456 789
Address: 123 Street, City, Country

CUSTOMER SERVICE

[Return & Delivery](#)
[Reviews](#)
[About us](#)

Share your experience with your friends on socials

POUŽÍVATEĽSKÝ PROFIL



Your orders



Wave rider Pro
Size: **S** | Amount: **1 pc**
Prize: **\$499.99**



BlueEcho
Size: **L** | Amount: **1 pc**
Prize: **\$783.00**



Maui vibes
Size: **M** | Amount: **1 pc**
Prize: **\$129.99**



Maui fast Fin
Size: **S** | Amount: **3 pc**
Prize: **\$329.97**

Alena

[LOG OUT](#)

[EDIT PROFILE](#)

[CHANGE PASSWORD](#)

Profile details
Email: a@a.com
Phone: 1234567890
Country: Slovensko





"You can't stop the waves but you can learn to surf"

CONTACT

Email: example@email.com
Phone: +123 456 789
Address: 123 Street, City, Country

CUSTOMER SERVICE

[Return & Delivery](#)
[Reviews](#)
[About us](#)

Share your experience with your friends on socials

KOŠÍK – 1. krok



Maui Surf



What's in my bag

①. step

②. step

③. step



Wave rider Pro

×

remove

S

1

Price: 499.99 \$



BlueEcho

×

remove

L

1

Price: 783.00 \$



Maui vibes

×

remove

M

1

Price: 129.99 \$



Maui fast Fin

×

remove

S

3

Price: 109.99 \$

Total: 1,742.95 \$

Continue Shopping

Checkout

KOŠÍK – 2. krok

CONTACT

Email: example@email.com
Phone: +123 456 789
Address: 123 Street, City, Country



Maui Surf



Fill out your information

①. step

②. step

③. step

Customer Information

Alena

Last Name

a@a.com

1234567890

Shipping Method

☒ Standard Shipping 3,99\$

☐ Express Shipping 6,99\$

Payment Method

☒ Card

☐ Apple Pay

☐ PayPal

☐ Google Pay

☐ Cash On Delivery

Billing Information

Town

Street

House Number

Region

Postal Code

Country

Shipping: 3.99\$

Total: 1746.94\$

Go Back

Continue

CONTACT

Email: example@email.com
Phone: +123 456 789
Address: 123 Street, City, Country

CUSTOMER SERVICE

Return & Delivery
Reviews
About us

Share your experience with your friends on
socials



KOŠÍK – 3. krok

The last step

①. step ----- ②. step ----- ③. step

Supported payment methods

Card Information

Shipping: 3.99\$

Total: 1746.94\$

Go Back

Purchase

CONTACT

Email: example@email.com
Phone: +123 456 789
Address: 123 Street, City, Country

CUSTOMER SERVICE

Return & Delivery
Reviews
About us

Share your experience with your friends on
socials

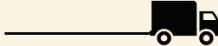
   

POTVRDENIE o objednaní

Thank you for your purchase!

You got a confirmation email
Your order is on its way to you!



Continue Shopping

CONTACT

Email: example@email.com
Phone: +123 456 789
Address: 123 Street, City, Country

CUSTOMER SERVICE

Return & Delivery
Reviews
About us

Share your experience with your friends on
socials

FAVORITES



Maui Surf

Your Favorite Products



Ocean pulse

\$1,199.99



Super fin

\$19.99



Surf Locos

\$29.99



Wave rider Pro

\$499.99

CONTACT

Email: example@email.com
Phone: +123 456 789
Address: 123 Street, City, Country

CUSTOMER SERVICE

Return & Delivery

Share your experience with your friends on socials

[f](#) [i](#) [l](#) [x](#)

ADMIN rozhranie



Products

Search...



ABC rider

Category: Surfboards
Subcategory: Short Surfboards
Price: \$199.99



StormSurge

Category: Surfboards
Subcategory: Mid-length Surfboards
Price: \$899.99



SunDrifter

Category: Surfboards
Subcategory: Mid-length Surfboards
Price: \$349.99



Reef Rider

Category: Surfboards
Subcategory: Mid-length Surfboards
Price: \$1,999.99



Ocean pulse

Category: Surfboards
Subcategory: Long Surfboards

ADMIN - maui surf

ADD PRODUCT

LOG OUT

Používateľská príručka (Windows / MacOS)

Táto príručka vás prevedie celým procesom úspešného spustenie nášho Laravel e-shop projektu. Nevyžaduje žiadne predchádzajúce nastavenie – každý krok je vysvetlený.

1. Predpoklady:

Je potrebné, aby ste pred spustením mali nainštalované tieto nástroje:

- PHP - minimálne verzia 8.1
- Composer
- PostgreSQL

2. Stiahnutie nášho projektu:

- Stiahnite si súbor nášho projektu z AIS – **WTECH_eshop.zip**
- Súbor môžete stiahnuť aj z nášho githubu:
<https://github.com/xmukova/WTECH-surf>
- Rozbaľte zip súbor

3. Otvorte priečinok v termináli:

```
cd moj-projekt
```

4. Nainštalujte si PHP závislosti:

```
composer install
```

5. Pripravte si súbor .env

- Skopírujte .env.example do .env

```
cp .env.example .env
```
- Upravte si ho podľa svojich nastavení

```
DB_CONNECTION=pgsql  
DB_HOST=127.0.0.1  
DB_PORT=5432 (alebo iný port v závislosti od vašej PostgreSQL konfigurácie)  
DB_DATABASE=databaza (alebo iný názov)  
DB_USERNAME=postgres  
DB_PASSWORD=heslo (heslo do PostgreSQL)
```

6. Vygenerujte aplikačný kľúč

```
php artisan key:generate
```

7. Spustite migrácie a naplňte databázu dátami:

```
php artisan migrate  
php artisan db:seed
```

8. Spustite server

```
php artisan serve
```

Prihlásanie ako Admin

Ak sa chcete prihlásiť do rozhrania pre Admina, prihlásenie je možné iba cez url:

<http://127.0.0.1:8000/login/admin>

Prihlasovacie údaje pre admina sú:

e-mail: `admin@mauisurf.com`

heslo: `heslokleslo`

Prihlásanie ako zákazník

Ak sa chcete prihlásiť do zákazníckeho účtu, je potrebné sa najprv registrovať, čím si vytvoríte vlastný účet.

Používateľská príručka na zakúpenie produktu

V košíku sme implementovali viacero validácií, ktoré musia byť splnené v každom kroku v košíku. V **prvom kroku košíka** je potrebné mať aspoň jeden produkt na to, aby bolo možné presunutie na druhý krok. V **druhom kroku košíka** musia byť všetky polia vyplnené a to tak, že **meno, priezvisko, mesto, ulica a región** musia obsahovať aspoň jedno písmeno, **číslo domu, PSČ, tel.č.** musia obsahovať iba čísla a email musí mať formát mailovej adresy.

Po výbere metódy platby **Cash on Delivery** sa preskakuje tretí krok košíka (vyplnenie karty), pretože sa platí na dobierku. V opačnom prípade v **treťom kroku košíka** je potrebné zadať v poli Name on Card aj meno aj priezvisko, ktoré musia obsahovať iba písmená. V poli Card Number je potrebné vložiť presne 16 číslíc. MM/YY reprezentuje dátum expirácie karty a túto validáciu sme tiež implementovali – vložené číslo musí byť validný dátum v budúcnosti. V poli CVC je potrebné vložiť 3 až 4 číslice.

Záver

V rámci semestrálneho projektu sme úspešne navrhli a implementovali vlastnú webovú aplikáciu - e-shop, ktorá spĺňa všetky požiadavky definované v zadaní. Projekt obsahuje plnohodnotnú klientsku časť pre bežného používateľa, ako aj administrátorskú zónu určenú na správu produktov. V aplikácii sme zabezpečili funkcionality ako prehľad produktov, filtrovanie, detail produktu, správa košíka, realizácia objednávky, registrácia a prihlásenie používateľa. Administrátor má možnosť produkty vytvárať, upravovať a mazať, vrátane nahrávania obrázkov.

Pri vývoji sme využili odporúčaný Laravel framework a PostgreSQL databázu, pričom sme dôsledne navrhli dátový model a zohľadnili princípy dobrej softvérovej architektúry. Projekt bol vypracovaný tímovo, pričom obe členky tímu sa aktívne podieľali na všetkých jeho častiach. Výsledná aplikácia plne zodpovedá požiadavkám zadania a je pripravená na rozšírenie alebo nasadenie v produkčnom prostredí. Našu priebežnú prácu sme počas implementácie projektu pridávali na [GitHub repozitár](#).

Dôvod 2. odovzdania

Projekt odovzdávame v druhom termíne z dôvodu technických problémov pri nahrávaní finálnej verzie do školského systému. V stanovenom termíne sme mali pripravenú kompletnú dokumentáciu, správu o GitHub repozitári a Laravel projekt bez priečinka `vendor`. Aj napriek odstráneniu tohto priečinka však projekt prekračoval maximálnu veľkosť pre nahranie do systému.

Preto sme sa rozhodli zmenšiť veľkosť projektu odstránením priečinka s obrázkami (`public/images`), pričom sme tieto súbory ponechali dostupné cez GitHub. Následne sa nám podarilo projekt nahrať, no systém zobrazoval chybu „**File not found**“. V snahe problém vyriešiť sme odstránili aj priečinok `storage`, pretože je to priečinok, ktorý neobsahuje súbory, kde vidno našu prácu, čo však viedlo k rovnakej chybe. Tesne pred uplynutím termínu sa nám síce podarilo súbor úspešne nahrať, no nahraná verzia neobsahovala teda priečinok `storage` a `images`.

Z tohto dôvodu odovzdávame projekt opätovne, naším cieľom je však odovzdať plne funkčnú a zrozumiteľnú verziu projektu, vrátane kompletného Laravel priečinka a všetkých potrebných súborov, hoci nás mrzí, že kvôli tejto komplikácii nám bude znížené hodnotenie, keďže naša celosemestrálna práca bola včas už v systéme nahratá. Odstránené zložky (`images`, `storage`) sú riadne prístupné v repozitári na GitHubu.